

Criterion A: Problem Specification

Problem:

The intended client for this project is the learner driver in Lithuania preparing for the national driving theory examination. The learner driver must memorise traffic rules and road signs and practise multiple-choice questions under exam-like conditions, with the goal of passing efficiently while developing a safe and responsible understanding of road use.

Many learners in Lithuania rely on textbooks or simple online quizzes. These methods often provide limited personalisation: questions repeat, revision is not structured by difficulty, and feedback is minimal. As a result, learners may spend time practising areas they already know while failing to identify weak topics, making revision inefficient and frustrating.

This problem was confirmed through an interview with a learner driver preparing for the driving theory test. During the interview, the client stated that questions in their current resources repeat frequently, that there is little feedback after answering questions, and that it is difficult to practise under realistic exam conditions.

We agreed that the learner driver needs an interactive practice system that presents theory-test-style questions in a structured way, prevents unnecessary repetition within a session, provides immediate feedback, and produces a clear end-of-session summary so the learner can plan targeted revision.

Rationale for the Proposed Solution:

The proposed solution is a **Driving Theory Practice Application** built as a desktop app using **JavaScript**. A desktop app was chosen because learner drivers typically revise on laptops, providing a consistent, privacy-preserving environment with reliable performance and simple local CSV file management for exam-style practice sessions. JavaScript was chosen because it allows the app to be interactive and easy to update. The app can run on **Windows** laptops. Performance issues are unlikely because the app works with local data and a manageable number of questions. The solution will be a **Client-side Web Application**. It will run locally in the browser to ensure privacy and zero latency, using the **File API** to read local CSVs without a backend server.

The **data environment** uses **CSV files** containing the question text, answer options, correct answers, and difficulty levels. CSV was chosen because it is easy to edit, scalable, and works well with JavaScript. All data will be stored locally, protecting user privacy.

These design choices ensure the system directly supports the learner driver's need for structured, exam-like practice with meaningful feedback.

Word count: 373

Success Criteria (functional) (CfS):

1. CSV question import: The system must load questions from a CSV file with the required fields (question text, four options A-D, correct option, difficulty). If any row is invalid, the system must skip it and report how many questions were loaded.
2. Randomised, non-repeating session: For a given session of questions, the system must present 30 unique questions selected from the available pool, in random order.
3. Difficulty handling and scoring: Each question must be tagged as Easy, Medium, or Hard, and the system must award 1 point for Easy, 2 points for Medium, and 3 points for Hard when answered correctly.
4. Immediate feedback: After each answer, the system must state whether the answer is correct and must display the correct option before moving on.
5. Accurate score calculation: The final score must equal the sum of points earned for all correctly answered questions, with no rounding errors.
6. Optional timing mode: If timing is enabled, the system must start timing at the first question and stop at completion, reporting the elapsed time to the nearest second.
7. End-of-session results: At completion, the system must display (a) total score, (b) number of correct/incorrect answers, and (c) a breakdown of performance by difficulty (e.g., correct out of total per level).
8. Session restart: The user must be able to start a new session without restarting the application, and the new session must reshuffle and reset all counters/timers.

Non-functional criteria:

- Performance: With a CSV bank of at least 500 questions, the application should (a) load the question bank in under 2 seconds on a typical school laptop and (b) respond to each submitted answer in under 0.2 seconds.
- Usability: A first-time user should be able to start a practice session within 30 seconds without external help. Instructions for answering (A-D) and the current question number must be clearly visible at all times.
- Reliability and error handling: The application must not crash if the CSV file is missing, or empty. Instead, it must display an actionable error message and allow the user to retry.
- Portability: The product must run on standard Windows laptops used by students.

- **Accessibility:** All core actions (start session, answer questions, move forward) must be usable via keyboard only. Text should be readable without requiring zoom.
- **Maintainability:** A non-technical user should be able to update the question set by editing/replacing the CSV file without modifying source code. The system should clearly document the required CSV format in the user guide.

Criterion B: Project Planning

Decomposing Problems

To ensure the Driving Theory Practice Application is developed logically, the project has been decomposed into modular components. Each module addresses specific success criteria (CfS) identified in Criterion A.

1. Data Ingestion & Validation

The system will use the **JavaScript File API** to read the user-provided CSV. The logic must parse the string data into an **Array of Objects**.

- **Validation Logic:** Before the session starts, a loop will check for the presence of mandatory fields (Question, A, B, C, D, Correct, Difficulty).
- **Error Handling:** If the file is empty or formatted incorrectly, a try...catch block will trigger a UI alert.
- **Supports:** *CSV question import, Reliability, Maintainability.*

2. Session Logic & Algorithmic Randomization

The system must generate a 30-question pool without repetition.

- **The Shuffle Algorithm:** I will implement the **Fisher-Yates Shuffle** on the master array to ensure true randomness before slicing the first 30 elements.
- **Edge Case Handling:** If the CSV contains <30 questions, the system will adapt by using the maximum available count rather than crashing.
- **Supports:** *Randomised session, Session restart.*

3. Dynamic UI & Navigation

The interface will be a **Single Page Application (SPA)** style, where the DOM is updated dynamically without refreshing the page.

- **Progress Tracking:** A global variable `currentQuestionIndex` will track progress and update the "Question X of 30" label.
- **Accessibility:** `event.key` listeners will be mapped to keys 'A', 'B', 'C', and 'D' to allow keyboard-only navigation.
- **Supports:** *Usability, Accessibility, Portability.*

4. Feedback & Scoring Engine

This module handles the real-time interaction logic.

- **Weighted Scoring:** A conditional check will apply the weight:

$$\text{Points} = (\text{Difficulty} == \text{'Hard'}) ? 3 : (\text{Difficulty} == \text{'Medium'}) ? 2 : 1.$$
- **Instant Feedback:** Upon selection, the "Correct Answer" class will be applied to the relevant button to highlight it green, providing the immediate visual feedback requested by the client.
- **Supports:** *Immediate feedback, Difficulty handling, Accurate scoring.*

5. Temporal Tracking

If the `timingEnabled` boolean is true, a `setInterval` function will trigger. The elapsed time will be calculated by comparing the `Date.now()` at start vs. finish to ensure accuracy to the nearest second.

- **Supports:** *Optional timing mode.*

6. Performance Analytics

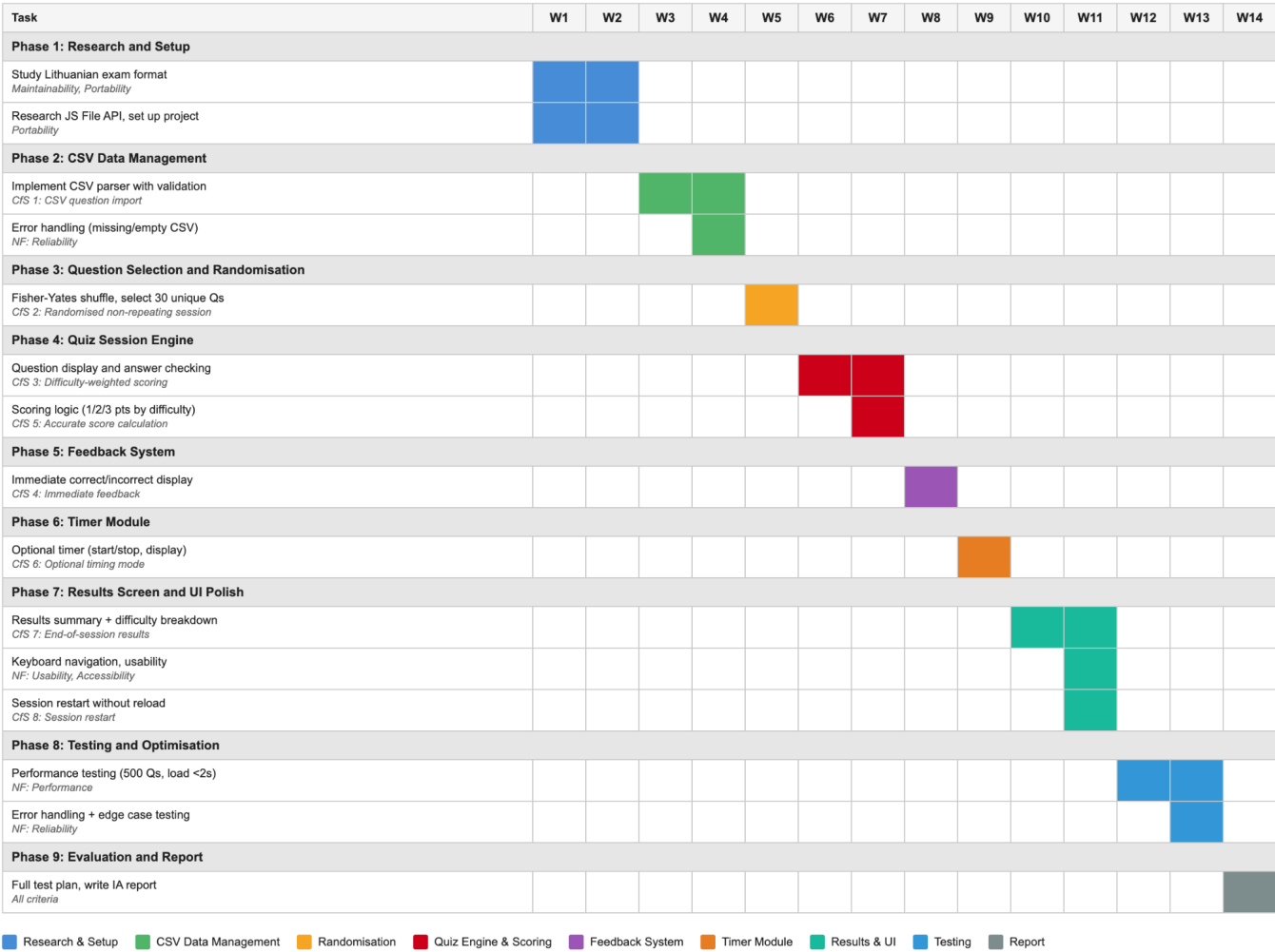
At the end of the session, the system will filter the results array to produce a breakdown.

- **Calculation:** $(\text{Correct} / \text{Total}) * 100$ per difficulty level.
- **Supports:** *End-of-session results.*

Planning the computational thinking process

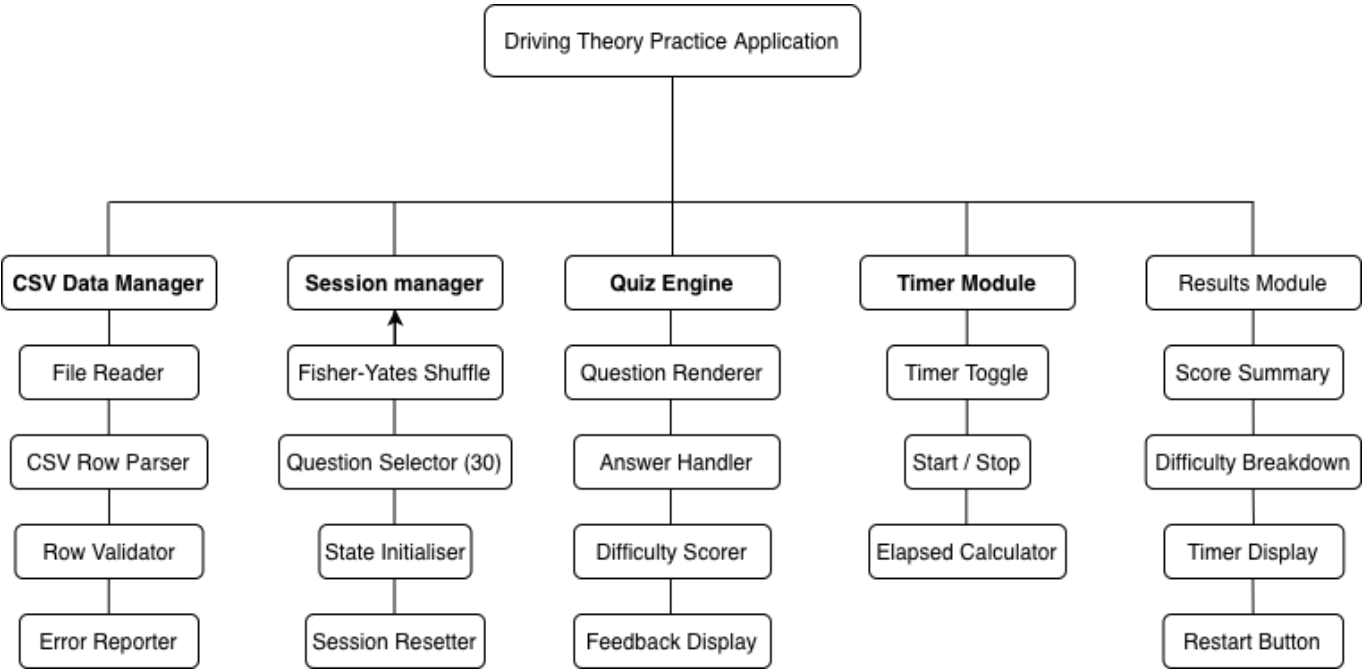
The development is scheduled over a 7-week period, following an Iterative Development approach. Each week concludes with a "Unit Test" to verify that the specific module meets its linked success criteria.

Figure 1: Gantt Chart - Driving Theory Practice Application



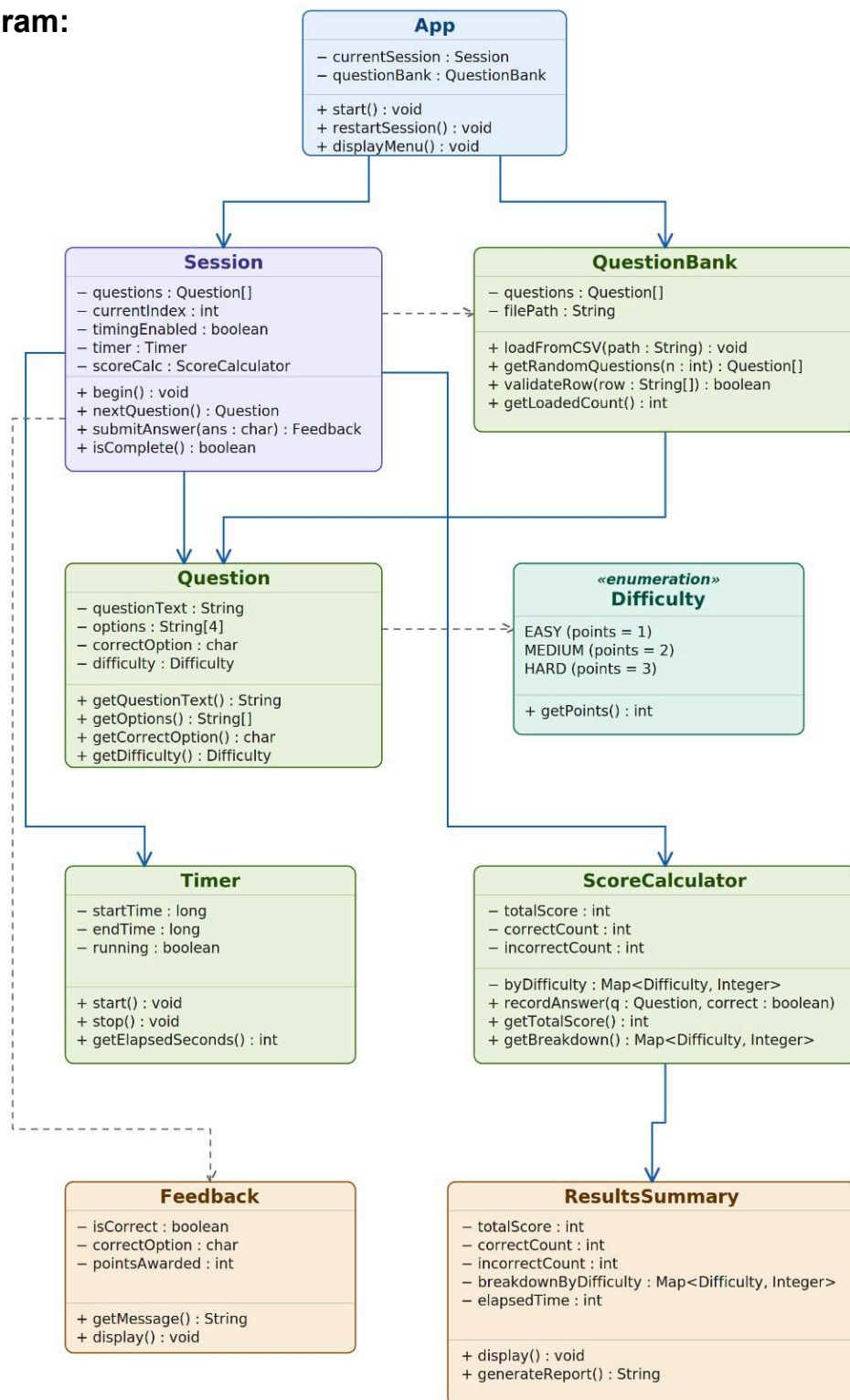
Criterion C - System Overview

Structure Chart:



The structure chart represents the modular decomposition of the system into smaller, manageable components. The top-level module controls overall program flow, while lower-level modules handle specific tasks such as user input, validation, processing, and output generation. The modular design was chosen to improve maintainability, reusability and readability.

UML Diagram:



Legend

→ Association / creates / populates

---> Dependency («uses» / produces)

□ Controller (App)

□ Session layer

□ Data / Utility

□ Enumeration

□ Output classes

Notation: - private, + public. Multiplicities shown near arrows (e.g. 1..*).

The UML Diagram illustrates the logical structure of the system by showing the main classes and their responsibilities. Each class encapsulates specific data and behaviors, following the principle of encapsulation.

Database Schema:

questions.csv (external file)	
question_text	STRING
option_a	STRING
option_b	STRING
option_c	STRING
option_d	STRING
correct_answer	CHAR (A B C D)
difficulty	ENUM (Easy Medium Hard)

SessionState (in-memory object)	
selectedQuestions	Question[30]
currentIndex	INTEGER (0-29)
score	INTEGER
correctCount	INTEGER
incorrectCount	INTEGER
difficultyStats	OBJECT
timerEnabled	BOOLEAN
startTime	NUMBER null
endTime	NUMBER null

This application does not use a relational database. All data comes from a CSV file read at session start. Session state is held in a JavaScript object in memory and is discarded when the page is closed. This design was chosen for simplicity, privacy, so no data leaves the client, and maintainability (anyone can edit the CSV).

Score Calculation Algorithm:

The system will calculate the user’s score based on their responses to each question. A conditional structure will be used to compare the user’s input with the correct answer. To enhance the complexity of the system, a scoring algorithm will be implemented that dynamically updates the user’s score. This will avoid a purely static approach and will allow the system to process multiple inputs efficiently. It will demonstrate computational thinking by using conditional logic and iterative processing to evaluate each response and calculate the final score.

GUI Wireframes:

The GUI wireframes show the layout and structure of the user interface. The design prioritizes simplicity and usability, ensuring that users can interact with the system without confusion. This

interface was designed to meet the client's requirement for an easy to use system. For example, buttons are clearly labeled and positioned consistently, allowing users to navigate the system intuitively. Users can also easily understand what actions to take. I also implemented input validation here to ensure the system doesn't crash if the user leaves a required field blank.

Driving Theory Practice

v1.0

Driving Theory Practice

Prepare for the Lithuanian driving theory exam with 30 randomised questions.

Step 1: Load your question bank

Choose CSV file...

✓ Loaded 500 questions (3 rows skipped)

Step 2: Options

✓ Enable timer

Start Session

Keyboard: Press Enter to start

Screen 1: Start / File Load

Timer: 04:32 | Difficulty: Medium (2 pts)

When approaching a pedestrian crossing where a pedestrian is waiting to cross, the driver must:

A) Speed up to pass before the pedestrian crosses

B) Stop and give way to the pedestrian

C) Sound the horn to alert the pedestrian

D) Continue at the same speed

Submit Answer

Keyboard: A / B / C / D to select, Enter to submit

When approaching a pedestrian crossing where a pedestrian is waiting to cross, the driver must:

A) Speed up to pass before the pedestrian crosses

B) Stop and give way to the pedestrian ✓

C) Sound the horn to alert the pedestrian

D) Continue at the same speed

Correct! +2 points

Next Question →

Keyboard: Enter or Space to continue

Screen 3: Feedback (Correct)

Session Complete

Time: 12:34

Results

42 / 60

70% - Good effort!

Correct answers	18 / 30
Incorrect answers	12 / 30
Time elapsed	12 min 34 sec

Performance by Difficulty

Easy (1 pt each): 8 / 10



Medium (2 pts each): 6 / 12



Hard (3 pts each): 4 / 8



Start New Session

Keyboard: Enter to restart

Screen 4: Results Summary

Flowcharts:

